

BINSON-SPEC-1.1

This is the complete specification of the Binson serialization format, version 1.1. Refer to it as **BINSON-SPEC-1.1**. Published October 2026.

1. INTRODUCTION

Binson is a simple, general-purpose data serialization format.

This specification describes the Binson object data structure and how it is serialized to bytes. Like objects of common programming languages, a Binson object has fields. A field is a named and typed value. There are seven value types: five primitive types (boolean, integer, double, string, bytes) and two composite types (array, object). An array is a finite sequence of unnamed, typed values.

2. FORMAT

The bytes of a serialized Binson object follow this [ABNF] syntax.

```
object      = begin *field end
field       = string value
value       = boolean / integer / double / string / bytes / array / object
array       = beginArray *value endArray
string      = stringLen utf
bytes       = bytesLen raw
boolean     = true / false
```

```
begin       = %x40
end         = %x41
beginArray  = %x42
endArray    = %x43
true        = %x44
false       = %x45
double      = %x46 float64
integer     = %x10 int8 / %x11 int16 / %x12 int32 / %x13 int64
stringLen   = %x14 int8 / %x15 int16 / %x16 int32
bytesLen    = %x18 int8 / %x19 int16 / %x1a int32
```

```
float64     = 80CTET ; double precision floating point number [IEEE-754]
int8        = 10CTET ; 8-bit signed two's complement integer
int16       = 20CTET ; 16-bit signed two's complement integer
int32       = 40CTET ; 32-bit signed two's complement integer
int64       = 80CTET ; 64-bit signed two's complement integer
utf         = *0CTET ; stringLen number of [UTF-8] bytes
raw         = *0CTET ; any sequence of bytesLen bytes
```

3. RULES

A finite sequence of bytes is a serialized Binson object if and only if the following rules are fulfilled.

1. The byte sequence must follow the format of the ABNF rule object.
2. Values must be stored using as few bytes as possible.
3. Fields must be stored in order. The order must be the lexicographical order of the [UTF-8] bytes of the name of the fields. Bytes are compared as unsigned numbers (0 to 255).
4. Two fields of the same direct parent object cannot have the same name.
5. Little-endian byte-order must be used.

4 . RECOMMENDATIONS

Non-normative recommendations:

1. An object should have less than 100 fields.
2. The size of a serialized Binson object should be less than 40 million bytes.
3. Field names should match the regular expression: `[a-zA-Z][a-zA-Z0-9_]{0,49}`.
4. Field names should use camel-case and start with a lower-case letter. Acronyms should be treated as words. Examples: `g8`, `httpHeader`, `customerId`.
5. It is recommended that a map (associative array) is stored as a single array. The order of the array values should be: key of first key-value pair, value of first key-value pair, key of second key-value pair, value of second key-value pair and so on.
6. A writer should store NaN as the bit pattern `0x7ff8000000000000` (a positive, quiet NaN), that is, as the bytes `46 00 00 00 00 00 00 f8 7f`.
7. Strings should be valid [UTF-8]. A Binson implementation need not check this. It can pass the bytes of a string on unchanged and leave UTF-8 handling, including validation, to the application. An implementation that converts strings to a native string type may reject or change strings that are not valid UTF-8.

The reasons for the recommendations are: 1. for readability and feasibility of linear search implementations, 2. for feasibility of in-memory processing, 3. for readability and interoperability with other object representations, 4. for consistency, 5. for consistency, 6. different programming languages produce different NaN bit patterns by default, and using one pattern gives the same bytes everywhere, 7. Binson treats a string as bytes marked as UTF-8 text. Checking and decoding the text belongs to the code that uses it, which keeps Binson implementations simple.

5 . VALUES

This section describes the values that the bytes represent.

- An integer is a signed 64-bit integer, from -2^{63} to $2^{63}-1$.
- A double is its 64-bit pattern, interpreted as an [IEEE-754] binary64 number.
- A string is a sequence of bytes marked as [UTF-8] text. Binson does not interpret the content of a string; the type is what distinguishes it from a bytes value.
- A bytes value is any sequence of bytes.

With these definitions, the rules of section 3 give each Binson object exactly one serialization.

6. REFERENCES

- **[UTF-8]** RFC 3629, *UTF-8, a transformation format of ISO 10646*, rfc-editor.org/rfc/rfc3629.
- **[IEEE-754]** IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019.
- **[ABNF]** RFC 5234, *Augmented BNF for Syntax Specifications: ABNF*, rfc-editor.org/rfc/rfc5234.
- **[FRANS]** Frans Lundberg, Stockholm, Sweden, franslundberg.com, [+46707601861](tel:+46707601861).

End of BINSON-SPEC-1.1.

The sections below are not part of the specification. They are non-normative and may be updated without a new version of the specification.

EXAMPLE

The object below, written in JSON-like notation,

```
{ "a": 1, "b": [true, 300], "c": "hi", "d": 1.5 }
```

is serialized to these 35 bytes (hexadecimal):

40		begin							
14	01	61	string "a"						
10	01		integer 1						
14	01	62	string "b"						
42			beginArray						
44			true						
11	2c	01	integer 300						
43			endArray						
14	01	63	string "c"						
14	02	68	69	string "hi"					
14	01	64		string "d"					
46	00	00	00	00	00	00	f8	3f	double 1.5
41									end

CHANGES FROM BINSON-SPEC-1

BINSON-SPEC-1.1 does not change the Binson format. It clarifies BINSON-SPEC-1 and fixes editorial errors. It updates references and adds the recommendations to serialize NaN values as `0x7ff8000000000000` and to use valid UTF-8 in strings. Version 1 is available as [BINSON-SPEC-1.pdf](#).

Copyright Frans Lundberg 2014-2026. The specification can be shared using the [CC BY-ND 4.0 licence](#).